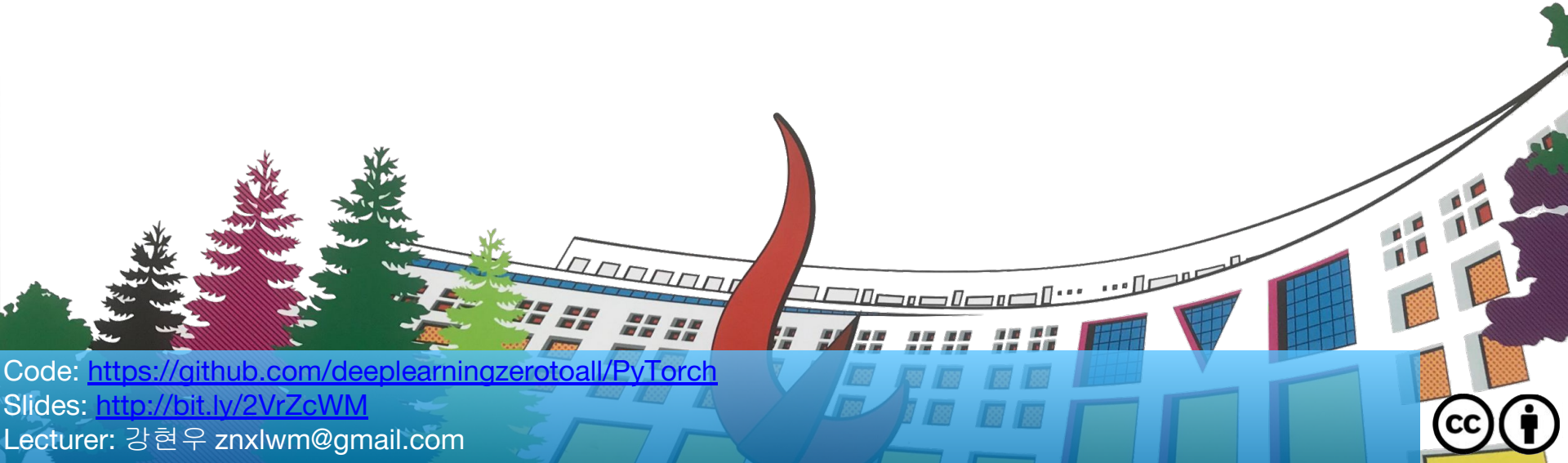
# ML/DL for Everyone Season2

## Batch Normalization



Code: https://github.com/deeplearningzerotoall/PyTorch
Slides: http://bit.ly/2VrZcWM
Lecturer: 강현우 znxlwm@gmail.com

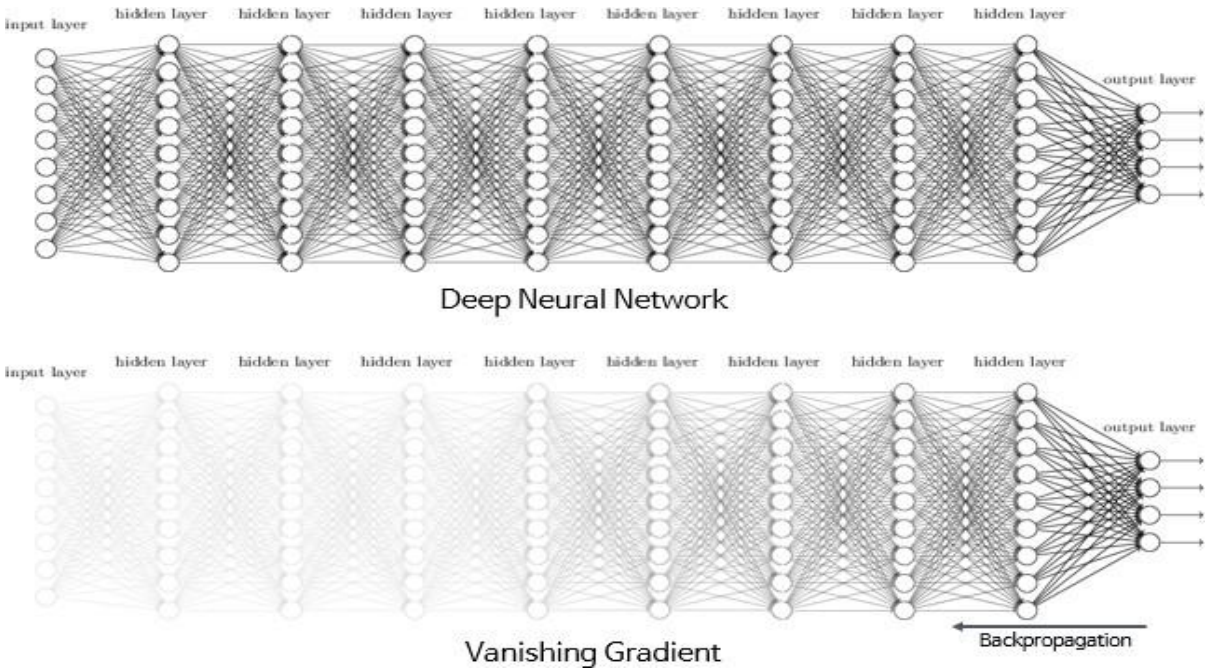
# Batch Normalization

- Gradient Vanishing / Exploding
- Internal Covariate Shift
- Batch Normalization
- Code: mnist_batchnorm

# Gradient Vanishing / Exploding

- Gradient Vanishing
- Gradient Exploding

# Gradient Vanishing / Exploding



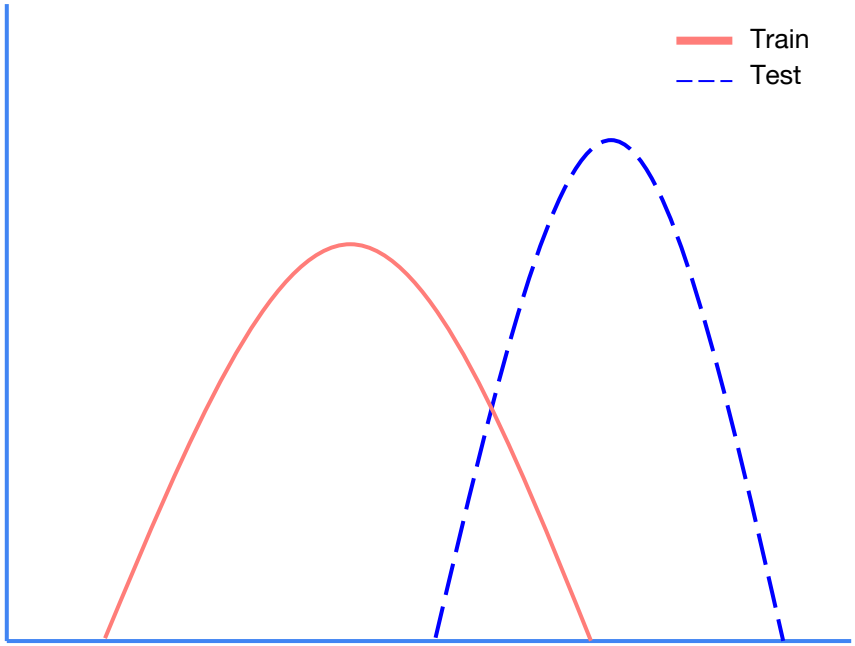
# Solution

- Change activation function
- Careful initialization
- Small learning rate

# 1. Activation 함수 변경
# 2. Weight Initialization 잘하기
# 3. Learning rate를 작게 설정하기

Batch Normalization !!

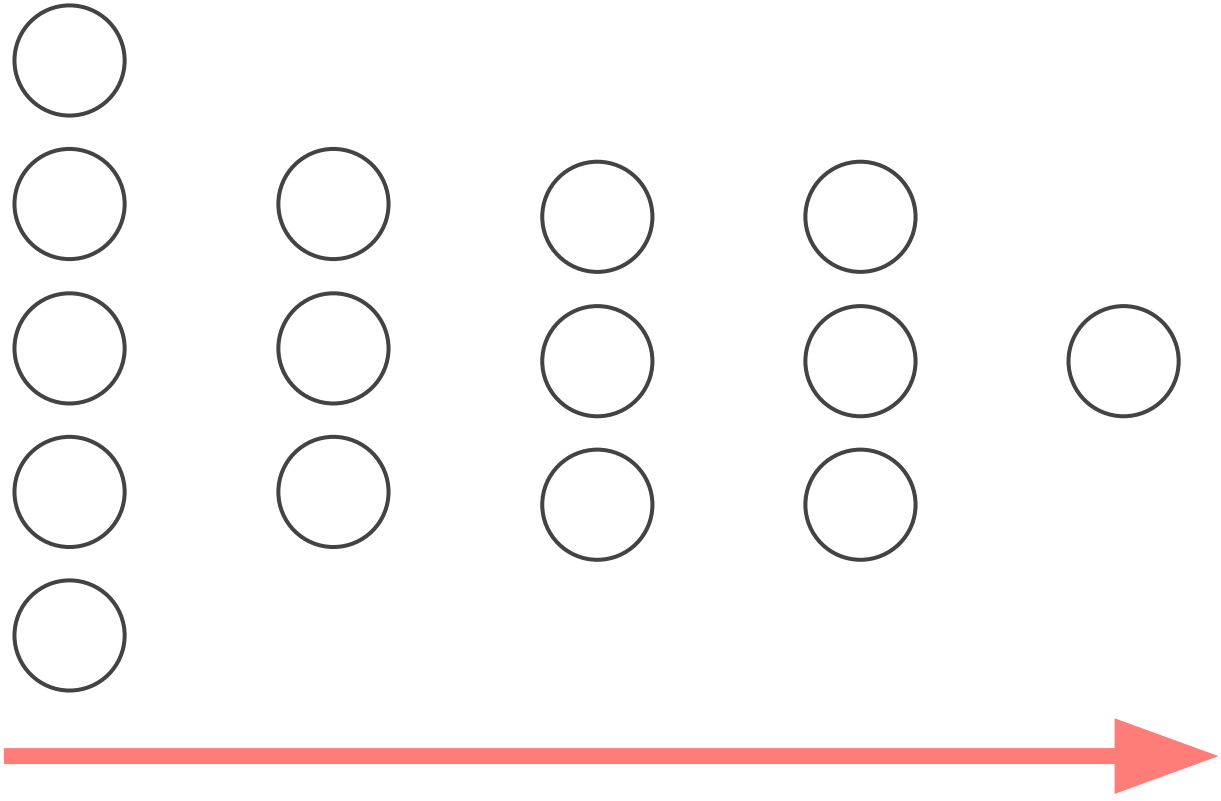
# Internal Covariate Shift




*Distribution의 차이
OOD 현상

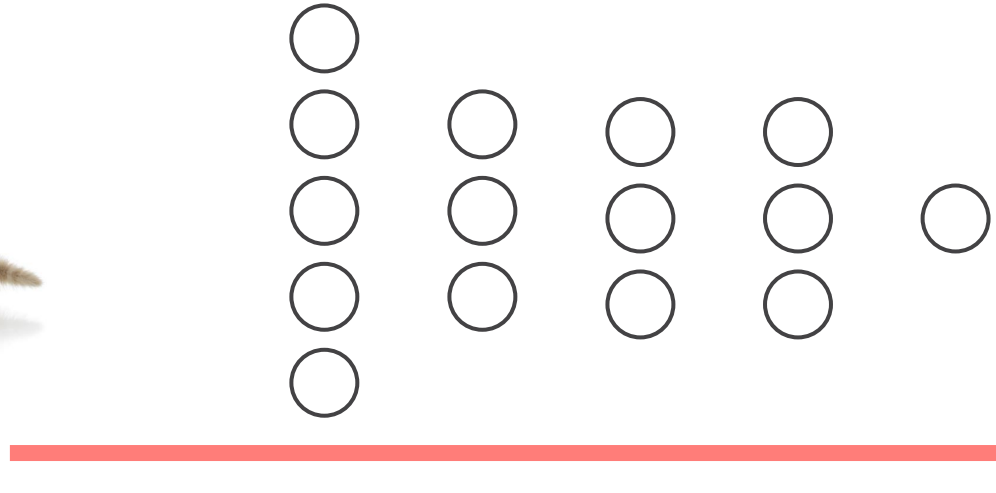
# Internal Covariate Shift





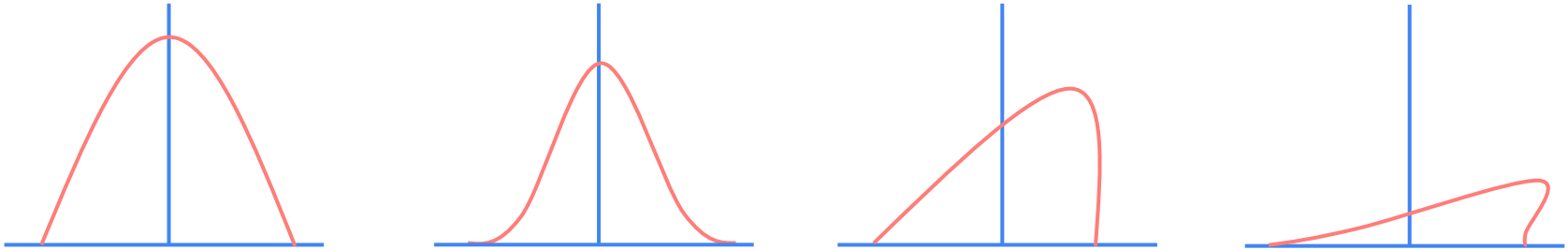
# Internal Covariate Shift



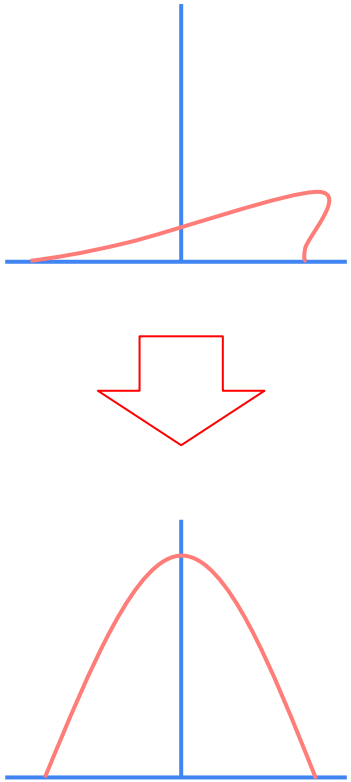


Cat

*Layer 각각의 단계 마다 생성



# Batch Normalization *Mini Batch마다 Normalization



**Input:** Values of $x$ over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: $\gamma, \beta$
**Output:** $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

배치의 데이터를 가져와서 평균과 분산 구하기
Test를 할때 고정된 값 사용

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

**Algorithm 1:** Batch Normalizing Transform, applied to activation $x$ over a mini-batch.

*배치마다 구했던 평균과 분산을 따로 저장해서 Test일때는 전체를 드러내는 값으로 하나로 쓰기

https://arxiv.org/pdf/1502.03167.pdf

# Train & eval mode

```
...
total_batch = len(data_loader)
model.train()    # set the model to train mode (dropout=True)
for epoch in range(training_epochs):
...

...
# Test model and check accuracy
with torch.no_grad():
   model.eval()    # set the model to evaluation mode (dropout=False)
...
```

**model.train() & model.eval()**

- Sets the module in training/evaluation mode.
- This has any effect only on certain modules. See documentations of particular modules for details of their behaviors in training/evaluation mode, if they are affected, e.g. `Dropout`, `BatchNorm`, etc.

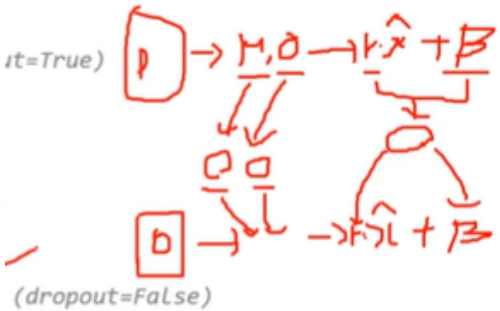
https://pytorch.org/docs/stable/nn.html?highlight=eval#torch.nn.Module.eval

# Code: mnist_batchnorm

```
# nn layers
linear1 = torch.nn.Linear(784, 32, bias=True)
linear2 = torch.nn.Linear(32, 32, bias=True)
linear3 = torch.nn.Linear(32, 10, bias=True)
relu = torch.nn.ReLU()
bn1 = torch.nn.BatchNorm1d(32)
bn2 = torch.nn.BatchNorm1d(32)

nn_linear1 = torch.nn.Linear(784, 32, bias=True)
nn_linear2 = torch.nn.Linear(32, 32, bias=True)
nn_linear3 = torch.nn.Linear(32, 10, bias=True)

# model
bn_model = torch.nn.Sequential(linear1, bn1, relu,
                               linear2, bn2, relu,
                               linear3).to(device)
nn_model = torch.nn.Sequential(nn_linear1, relu,
                               nn_linear2, relu,
                               nn_linear3).to(device)
```




*같은 u와 v의 값을 쓰기 때문에
batch normalization 문제를 해결 할 수 있음

*배치마다 구한 running mean과 variance를 반영
해서 하나의 수식으로 귀결하게 만들기

# Code: mnist_batchnorm

```
for epoch in range(training_epochs):
    bn_model.train()  # set the model to train mode

    for X, Y in train_loader:
        # reshape input image into [batch_size by 784]
        # label is not one-hot encoded
        X = X.view(-1, 28 * 28).to(device)
        Y = Y.to(device)

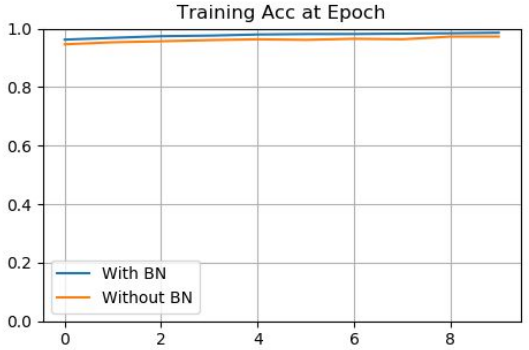
        bn_optimizer.zero_grad()
        bn_prediction = bn_model(X)
        bn_loss = criterion(bn_prediction, Y)
        bn_loss.backward()
        bn_optimizer.step()

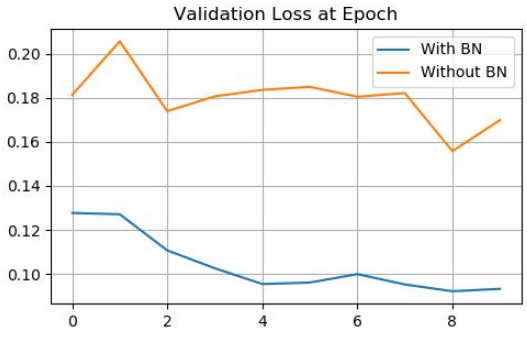
        nn_optimizer.zero_grad()
        nn_prediction = nn_model(X)
        nn_loss = criterion(nn_prediction, Y)
        nn_loss.backward()
        nn_optimizer.step()
```

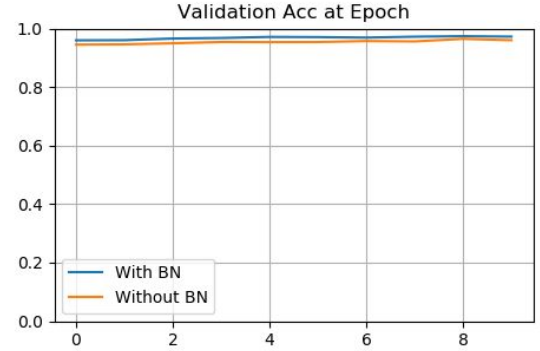
*train 모드 선언 필수

# Code: mnist_batchnorm



# What’s Next?

- Convolutional Neural Network

학습 때는 매번 미니배치의 평균($\mu_B$)과 분산($\sigma_B^2$)을 계산해서 뺐지만, **테스트 때는 계산을 생략하고 그 자리에 고정 통계량을 그대로 집어넣습니다.**

$$\hat{x}_{test} = \frac{x_{test} - \text{running_mean}}{\sqrt{\text{running_var} + \epsilon}}$$